

Week 03 — Monte Carlo and Covariance Estimation

FinTech 545 — Quantitative Risk Management

Dominic Pazzula

Table of contents

1	Monte Carlo Simulation	2
1.1	The Inverse CDF Method	2
1.2	Simulating a Correlated System	3
2	Cholesky Factorization	4
2.1	The Algorithm	4
2.2	The PSD Case	6
3	Missing Data	6
4	Covariance Matrix Generation Techniques	7
5	Exponentially Weighted Covariance	8
5.1	The Estimator	8
5.2	Choosing Lambda	9
5.3	Effective Sample Size	10
5.4	Speed of Update	11
5.5	The Link to GARCH	12
6	Dealing with Non-PSD Matrices	12
6.1	Rebonato and Jackel	13
6.2	Higham	14
6.3	In Practice	15
7	Principal Component Analysis	15
7.1	The Decomposition	16
7.2	Dimensionality Reduction	17
7.3	Variance Explained	17
8	Simulating from Models	17

1 Monte Carlo Simulation

The Monte Carlo method is something that is used widely throughout risk management. Often we are dealing with functions of random variables, from which the final distribution of outcomes is unknown. The Monte Carlo method gives us a way to approximate that final distribution.

Week 01 and Week 02 gave us distributions we can write down. Almost nothing we actually hold is one of them. A portfolio of options on correlated assets has a P&L distribution with no closed form, and no amount of algebra will produce one. Simulation replaces the algebra. Draw the inputs from a distribution we do understand, push each draw through the function, and read the answer off the resulting sample.

The cost is that the answer is itself a random variable. Every number produced this way carries simulation error, and that error falls as $1/\sqrt{K}$ in the number of draws.

1.1 The Inverse CDF Method

Drawing univariate random numbers is easy. Most software packages have a random function and many will allow you to specify a distribution.

Sometimes, only the uniform distribution is available from the random number generator, defined on the interval $(0,1)$. Using the quantile function, the inverse CDF, we can use that to give us random draws from a distribution.

Quantile function is the inverse of the CDF. It is the value of x such that $F(x) = p$:

$$F^{-1}(p)$$

Remember the CDF function is

$$F(x) = P(X \leq x) = \int_{lb}^x f(t) dt$$

where lb is the lower bound of the distribution.

The CDF is monotonic and defined between $(0,1)$, so the quantile function gives a 1:1 mapping from $(0,1)$ into the range of the distribution.

$$x = F^{-1}(u), \quad u \sim U(0,1)$$

This one line does more work than it appears to. It converts any distribution with a computable quantile function into a draw, using only a uniform generator. It is also the mechanism behind the copula in Week 05, where we run the transform in both directions – CDF to get to uniforms, inverse CDF to get back out.

1.2 Simulating a Correlated System

Often we need to simulate a system of correlated values. Most commonly, we use a multivariate normal distribution.

Recall last week we stated if X was multivariate normal

$$X \sim N(\mu, \Sigma)$$

There exists

$$\mu \in \mathbb{R}^n, L \in \mathbb{R}^{n \times m} \quad \text{such that} \quad X = LZ + \mu$$

Where

$$Z_i \sim N(0, 1), \quad i \in [1, m]$$

$$\Sigma = LL'$$

This provides a path for the generation of multivariate random normals.

Multivariate Draw Algorithm. To simulate K random draws from an N variable multivariate normal, we need:

1. Factor Σ into L , which is $(N \times M)$
2. Generate $K \times M$ random standard normals, Z , shaped $(M \times K)$
3. $X = (LZ + \mu)^T$, with shape $(K \times N)$

Each series is on the row without the transpose. We tend to think of tables with columns of values, so we use the transpose to put the matrix into a data table format.

A lot of packages have a multivariate normal random number generator. However, with some of the data problems we will encounter, knowing how to generate your own will be necessary.

The rest of this week is about step 1. Steps 2 and 3 are a random number call and a matrix multiply. Producing an L from a covariance matrix estimated on real data is where the work is, and where the failures are.

2 Cholesky Factorization

The most used factorization of Σ is the Cholesky root. The Cholesky factorization can be applied to positive definite (PD) matrices. The factor L is a lower triangular matrix. This is sometimes referred to as the Cholesky root, as the factor is analogous to the square root.

The Cholesky root is roughly 2x more efficient to calculate than the LU factorization. For this reason it is more often used.

If the matrix is only positive semi-definite (PSD), the Cholesky root can still be calculated allowing for a column of 0 values. The number of non-zero columns aligns with the number of positive eigenvalues (the rank) of the matrix.

2.1 The Algorithm

A is symmetric and PD. L is lower triangular, and $A = LL'$. Multiply the two triangular matrices and match entries, and every element of L falls out in terms of elements already computed:

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=1}^{j-1} L_{jk}^2}$$
$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=1}^{j-1} L_{ik} L_{jk} \right), \quad i > j$$

Nothing on the right hand side of either expression involves an unknown, which is why a single pass works.

This is solved either across the rows or down the columns.

Down the columns:

1. Column j , start on the diagonal element
2. Subtract the sum of the squares of the values on the root matrix for row j from the value on the input matrix on the diagonal
3. Update the root matrix at position (j,j) with the square root of step 2
4. Moving down the column, row i :
 - a. Calculate the dot product of sub matrix vector $[i, 1:(j-1)]$ and $[j, 1:(j-1)]$
 - b. Subtract a. from the (i,j) element of the input matrix
 - c. Divide b. by the j diagonal element of the root matrix
 - d. Store that value in element (i,j) of the root matrix
5. Repeat for the next column

2.1.1 A Worked Example

Take the symmetric PD matrix

$$A = \begin{pmatrix} 4 & 2 & -2 & 4 \\ 2 & 10 & 5 & -1 \\ -2 & 5 & 21 & 0 \\ 4 & -1 & 0 & 15 \end{pmatrix}$$

Read as a covariance matrix, the variances are 4, 10, 21, and 15, and the implied correlations run from -0.22 to 0.52 .

Column 1. The diagonal has no prior terms to subtract, so $L_{11} = \sqrt{4} = 2$. Divide the rest of the column by it:

$$L_{21} = \frac{2}{2} = 1, \quad L_{31} = \frac{-2}{2} = -1, \quad L_{41} = \frac{4}{2} = 2$$

Column 2. Subtract the squares already placed in row 2, then divide:

$$L_{22} = \sqrt{10 - 1^2} = \sqrt{9} = 3$$

$$L_{32} = \frac{5 - (-1)(1)}{3} = \frac{6}{3} = 2, \quad L_{42} = \frac{-1 - (2)(1)}{3} = \frac{-3}{3} = -1$$

Column 3.

$$L_{33} = \sqrt{21 - (-1)^2 - 2^2} = \sqrt{16} = 4$$

$$L_{43} = \frac{0 - (2)(-1) - (-1)(2)}{4} = \frac{4}{4} = 1$$

Column 4. Only the diagonal is left:

$$L_{44} = \sqrt{15 - 2^2 - (-1)^2 - 1^2} = \sqrt{9} = 3$$

Collecting the results:

$$L = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 1 & 3 & 0 & 0 \\ -1 & 2 & 4 & 0 \\ 2 & -1 & 1 & 3 \end{pmatrix}$$

Multiply LL' and confirm you recover A . Work this by hand once. Every failure mode discussed below is a failure of one of the four square roots or one of the six divisions above.

Most linear algebra packages use this algorithm on blocks of the matrix. The dot products become matrix multiplications, matrix inverse for the division, and the square root is a single threaded Cholesky call. The block size is chosen based on CPU cache size and vector operations are used for the matrix multiplications. See <http://www.netlib.org/utk/papers/factor/node9.html>.

2.2 The PSD Case

If the matrix is PSD instead of PD, the algorithm above breaks down. At least one of the root diagonal elements will be 0. The calculation of the column then fails when we attempt to divide by 0. In this case, we set all the values of the column to 0.

Two things follow from that patch, and both matter later.

The first is that it works. LL' still reproduces Σ exactly, and the simulation still runs. The Cholesky failing is a property of the PD algorithm, not of the matrix.

The second is that it is not free. A zero column means one fewer independent random number driving the system. Simulating N variables from a rank M matrix gives draws that all lie inside an M dimensional subspace, and no simulated scenario can ever produce a pattern outside it. Week 09 is built on that observation.

3 Missing Data

Missing values are very common in finance. Not all markets are open at the same time on the same days. A holiday in one market is not necessarily a holiday in another, even in the same country.

Imagine 3 assets with different trading schedules. Table 1 shows the days on which a market is open and the asset trades.

Table 1: Three assets on different trading calendars

Date	Asset A	Asset B	Asset C
1	X	X	
2	X	X	X
3		X	X
4			
5	X		
6	X	X	X
7	X	X	X
8	X	X	
9	X	X	X
10		X	X

2 common ways to calculate (we will talk about some others later):

1. Only use the days on which all markets are open.
 - a. This limits the information

- b. Only days 2, 6, 7, and 9 (40%) are open at the same time
 - c. Information about asset behavior on the other days is ignored
- 2. Use pairwise calculations. Find the matching rows for each pair, and build the covariance matrix piece by piece.
 - a. We use more information, but not all
 - b. Assets A and B would use days 1, 2, 6, 7, 8, and 9
 - c. While we are building a symmetric matrix, we do not guarantee the matrix is PSD

The trade in Table 1 is a real one. Method 1 keeps 4 rows out of 10 and returns a matrix that is guaranteed PSD. Method 2 uses 6 rows for the A-B pair and returns a matrix that may not be. Each entry of a pairwise matrix is estimated from a different sample, so the entries are not required to be mutually consistent, and a set of mutually inconsistent correlations is exactly what a negative eigenvalue is.

Nothing here is a rounding problem. Section 6 is about what to do once it happens.

4 Covariance Matrix Generation Techniques

There are many different techniques people use in practice for creating covariance matrices.

1. Default covariance function. With perfect data, and an assumption of normality, then the covariance as we described last class is the best estimator.
2. If there are missing values that do not fully overlap, but enough history that does, then the method of using only records with overlapping data works well. It might not have the full information, but because you have a lot of data, the matrix should be OK.
3. If you are missing values and do not have a lot of fully overlapping observations, then pairwise estimation can be used. This will often result in a non-PSD matrix, which we will discuss shortly.
4. Financial data are time variant. Often we use a weight function to give more weight to recent observations – usually with weights decreasing exponentially. This is sometimes called the RiskMetrics covariance, after the firm that popularized it in the mid 1990s. Section 5 details the calculation.
5. Another popular method is using an EM algorithm to estimate the covariance. Expectation Maximization is an iterative process using the likelihood function. This can deal with the missing data problem. However, as the proportion of missing values as a percent of the total observations increases, the results can be unstable. It is best suited to a large number of observations so that convergence can occur.
 - a. We will not go into detail, but I would highly recommend looking into this.
6. When dealing with high frequency data, the issue of overlapping observations becomes even harder. With time intervals measured in milliseconds, it is impossible to see updated prices on all assets. Lots of work has gone into this area recently. There are a few packages, “highfrequency” in R and “HighFrequencyCovariance” in Julia.
 - a. If you are going into the field of automated trading and will be dealing with very short time interval data, start by reading the paper for “HighFrequencyCovariance” in Julia and follow the references. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3786912

- b. If someone knows of an analogous package in Python, please let me know and I will add it to the course notes.
- 7. Because the issue of rank of the matrix comes from the correlation estimation, we will use any of the methods in this list to calculate correlations, but use the full history of each variable for the variance calculation.

Item 7 is worth reading twice. Splitting the problem into a correlation matrix and a vector of variances lets each piece use the data it can, and it isolates the rank problem in the correlation matrix where the repair algorithms in Section 6 operate. Rebuild with

$$\Sigma = D C D, \quad D = \text{diag}(\sigma_1, \dots, \sigma_n)$$

so a full-history variance for each variable survives whatever was done to C .

When we break the assumption of normality and we have what look like outliers to the normal distribution, the Pearson coefficient may not be the best estimator of correlation. We will discuss this in a few weeks.

5 Exponentially Weighted Covariance

JP Morgan was one of the first banks to use exponentially weighted covariance matrices in their risk analytics starting in the late 1980s. Their risk management was considered some of the best in the industry, and they spun out a software and consulting company called “RiskMetrics.”

5.1 The Estimator

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda) (x_{t-1} - \bar{x})^2$$

This is a simple exponential smoothing model. The variance for tomorrow is updated based on today’s forecast variance and today’s deviation from the mean.

Back substitute and we get

$$\sigma_t^2 = (1 - \lambda) \sum_{i=1}^{\infty} \lambda^{i-1} (x_{t-i} - \bar{x})^2$$

Recognize that

$$\lim_{n \rightarrow \infty} (1 - \lambda) \sum_{i=1}^n \lambda^{i-1} = 1$$

Setting the weight of a prior observation

$$w_{t-i} = (1 - \lambda) \lambda^{i-1}$$

will give us a weighted estimator over an infinite horizon. Obviously we do not have an infinite horizon, so weights are normalized so that they sum to 1:

$$\hat{w}_{t-i} = \frac{w_{t-i}}{\sum_{j=1}^n w_{t-j}}$$

Variance and covariance estimators are then

$$\widehat{\sigma_t^2} = \sum_{i=1}^n \hat{w}_{t-i} (x_{t-i} - \bar{x})^2$$

$$\widehat{\text{cov}(x, y)} = \sum_{i=1}^n \hat{w}_{t-i} (x_{t-i} - \bar{x}) (y_{t-i} - \bar{y})$$

Equal weighting the time series reduces to the MLE values.

5.2 Choosing Lambda

RiskMetrics research found that across a number of asset classes, $\lambda = 0.97$ was the best value.

Be careful with that number. 0.97 is the RiskMetrics decay factor for monthly data. For daily data, which is what we use in this class, RiskMetrics used $\lambda = 0.94$.

Treat both as starting guides rather than settings. They came from one firm's data in the mid 1990s, across a particular set of asset classes, at a particular sampling frequency. A more liquid market, a longer horizon, or a different use for the estimate can all justify moving off them. What the choice should not be is unexamined.

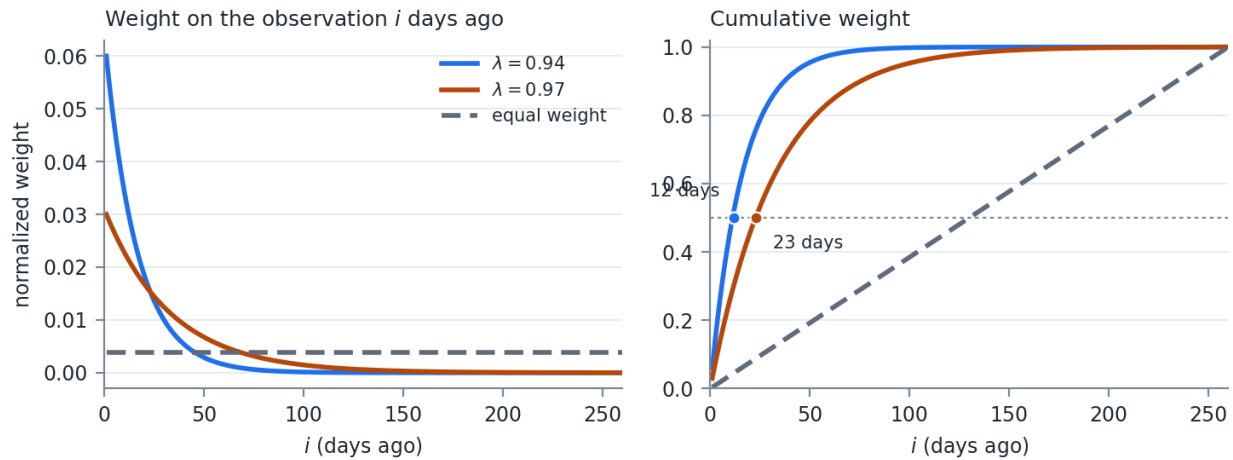


Figure 1: Normalized exponential weights over one trading year, against equal weighting

Figure 1 shows what the choice buys and what it costs. At $\lambda = 0.94$, half the total weight sits in the most recent 12 observations. At $\lambda = 0.97$ it takes 23. Equal weighting over the same 260 days puts half the weight in the most recent 130.

5.3 Effective Sample Size

An equally weighted estimator built on n observations uses n observations. An exponentially weighted one does not, and the number it does use is worth computing.

The standard summary is the effective sample size:

$$n_{\text{eff}} = \frac{1}{\sum_i \hat{w}_i^2}$$

For equal weights, $\hat{w}_i = 1/n$, the sum is $1/n$, and $n_{\text{eff}} = n$. For exponential weights on an infinite horizon the sum has a closed form:

$$\sum_{i=1}^{\infty} \hat{w}_i^2 = (1 - \lambda)^2 \sum_{i=1}^{\infty} \lambda^{2(i-1)} = \frac{(1 - \lambda)^2}{1 - \lambda^2} = \frac{1 - \lambda}{1 + \lambda}$$

$$n_{\text{eff}} = \frac{1 + \lambda}{1 - \lambda}$$

The half life follows from the same weights:

$$h = \frac{\ln(0.5)}{\ln(\lambda)}$$

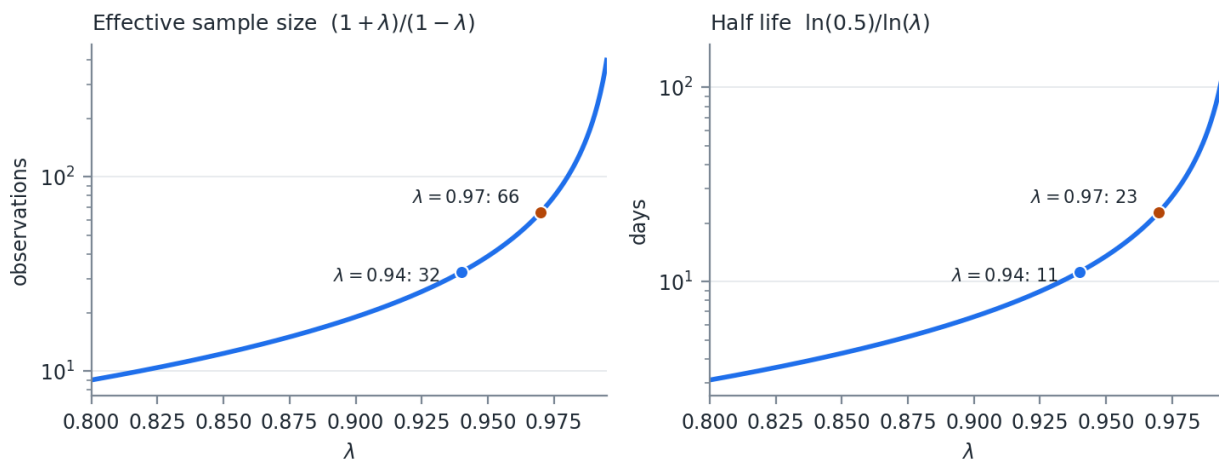


Figure 2: Effective sample size and half life across the usable range of λ

Three things follow from Figure 2.

n_{eff} does not depend on n . At $\lambda = 0.94$ it is 32 observations whether you feed the estimator one year of history or ten. Adding data past roughly four half lives changes the estimate by less than the rounding on the way to the report. Storing five years of returns to run a $\lambda = 0.94$ estimator is storage, not information.

The curve is convex, and it gets steep exactly where people set λ . Moving from 0.94 to 0.97 doubles n_{eff} from 32 to 66. Moving from 0.97 to 0.99 triples it again to 199. Small changes in λ are not small changes in the estimator, and reporting λ to two decimals conceals that.

Estimation error scales with n_{eff} , not n . The standard error of a volatility estimate is roughly $\sigma/\sqrt{2n_{\text{eff}}}$, which is about 12.5% of the estimate at $\lambda = 0.94$ and 8.7% at $\lambda = 0.97$. That is the price of responsiveness, and it is not small. A 20% volatility estimated at $\lambda = 0.94$ carries a standard error of about 2.5 volatility points.

There is also a conditioning consequence. Every weight is strictly positive, so the matrix is not singular in exact arithmetic. But the smallest eigenvalues are supported almost entirely by observations carrying vanishing weight, so a covariance matrix estimated at $\lambda = 0.94$ across more than roughly n_{eff} assets is numerically close to singular. Section 6 exists partly because of this.

My opinion: on daily data, use at least a year of history. Then be honest with yourself that the estimate is built on a few weeks of it.

5.4 Speed of Update

The other half of the trade is how fast the estimate moves.

The recursion gives the answer directly. Today's observation enters σ_t^2 with weight exactly $(1 - \lambda)$ – 6% at $\lambda = 0.94$, 3% at $\lambda = 0.97$, 1% at $\lambda = 0.99$. Everything already in the estimate is scaled by λ . A shock that entered k days ago retains λ^k of the weight it arrived with, so it decays on the same half life as Figure 2.

Put a 5 standard deviation day through it. The squared observation is $25\sigma^2$, so:

$$\sigma_{\text{new}}^2 = \lambda\sigma^2 + (1 - \lambda) 25\sigma^2 = \sigma^2 (\lambda + 25(1 - \lambda))$$

Table 2: How λ controls the response to a single large move

λ	Weight on today	Volatility after a 5σ day	Half life
0.90	10%	$\times 1.87$	7 days
0.94	6%	$\times 1.56$	11 days
0.97	3%	$\times 1.31$	23 days
0.99	1%	$\times 1.11$	69 days

Table 2 is the whole trade in one place. The same event moves a $\lambda = 0.90$ estimate by 87% and a $\lambda = 0.99$ estimate by 11%, and the faster estimate then forgets it ten times sooner.

Neither end is right. A fast λ tracks a regime change within a week and produces limit breaches on days when nothing happened. A slow λ is stable and will still be reporting last month's calm in the middle of a crisis. Pick based on what the number is for. A risk limit that triggers trading wants responsiveness. A capital calculation wants stability.

Whatever you pick, say what it is. An unstated λ makes two volatility numbers incomparable even when they were computed from the same returns.

5.5 The Link to GARCH

Remember the GARCH(p,q) model from last week:

$$\sigma_t^2 = \omega + \sum_{i=1}^p (\alpha_i \epsilon_{t-i}^2) + \sum_{i=1}^q (\beta_i \sigma_{t-i}^2)$$

If we set $(p, q) = (1, 1)$ then we can see RiskMetrics' exponentially weighted methodology is a GARCH(1,1) process with $\omega = 0$, $\alpha = (1 - \lambda)$, and $\beta = \lambda$.

We also said that if $\alpha + \beta = 1$ the long term variance is undefined. Do not use long time horizon simulation where you update the covariance matrix for each future time period using this methodology.

The reason is in the forecast recursion from last week. With $\alpha + \beta = 1$ there is no level to revert to, so the k step ahead forecast is just today's variance repeated forever, and any simulation that keeps updating will wander without bound.

The missing long run variance is not only a liability. GARCH with $\alpha + \beta < 1$ mean reverts to a fixed level, so when volatility shifts to a genuinely new regime the model fights the shift – the forecast keeps decaying back toward the old level. The exponentially weighted estimator has no old level to decay toward, so it absorbs the shift permanently. Clustered spikes are what GARCH was built to describe. Persistent level shifts are what the exponentially weighted estimator handles better.

Lamoureux and Lastrapes (1990) make the point from the other direction. Ignoring structural breaks in the unconditional variance biases the estimated $\alpha + \beta$ upward toward 1, so the persistence of 0.95 to 0.99 typically fitted on daily equity data is partly an artifact of regime shifts the model does not represent.

Be careful about how much this buys. Neither estimator observes the regime. Both learn only through squared returns, at speed α for GARCH and $(1 - \lambda)$ for the exponentially weighted estimator, and typical values of those are 0.05 to 0.10 against 0.06. The advantage is in the forecast, not in the detection. The same refusal to revert also means a one off spike stays in the estimate long after it should have gone.

6 Dealing with Non-PSD Matrices

2 main reasons matrices are non-PSD:

1. Floating point error
2. Missing data

Floating point error and rounding can become a problem when calculating the Cholesky on a PSD matrix. Sometimes you have a true PSD matrix but you end up with a slightly negative eigenvalue. That will cause one of the diagonals on the root matrix to be complex. Generally it is OK to check for and set to 0 slightly negative values before you attempt the square root. Negative values too large to be floating point error need to be dealt with in a different manner.

The distinction is one of magnitude, and there is no clean threshold. An eigenvalue at -10^{-16} times the largest eigenvalue is arithmetic. An eigenvalue at -0.05 times the largest is a statement that the data are mutually inconsistent, and zeroing it silently changes the matrix.

6.1 Rebonato and Jackel

Rebonato and Jackel (1999, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=1969689) present 2 methods for finding an acceptable PSD matrix. The method using a spectral decomposition on the correlation matrix is relatively straightforward.

First, decompose the correlation matrix:

$$CS = S\Lambda, \quad \Lambda = \text{diag}(\lambda_i)$$

This is the eigenvalue decomposition of C into its eigenvectors S and a diagonal matrix with its eigenvalues, Λ .

Second, construct a new Λ' such that all eigenvalues are greater than or equal to 0:

$$\Lambda' : \lambda'_i = \max(\lambda_i, 0)$$

Third, construct the diagonal scaling matrix

$$T : t_i = \left[\sum_{j=1}^n s_{i,j}^2 \lambda'_j \right]^{-1}, \quad T = \text{diag}(t_i)$$

$s_{i,j}$ is the (i, j) element of the eigenvector matrix. T is what restores the unit diagonal. Zeroing eigenvalues shrinks the rows, and without the rescaling the result would no longer be a correlation matrix.

Now let

$$B = \sqrt{T} S \sqrt{\Lambda'}$$

Where $\sqrt{\cdot}$ of a matrix denotes the element wise square root.

$$BB^T = \hat{C} \approx C$$

This matrix is not necessarily the closest matrix (by whatever method of “closest” you choose) to the original. It is, however, a good starting point and oftentimes, good enough.

Rebonato and Jackel also present an optimization algorithm in their paper, which we will not discuss here.

6.2 Higham

Higham (2002, <https://www.maths.manchester.ac.uk/~higham/narep/narep369.pdf>) presents an iterative algorithm for finding the nearest PSD correlation matrix. This method is used widely.

The goal is to find A such that

$$\gamma(A) = \|A - C\|$$

is minimized, such that A is PSD.

Generally, the norm used is the Frobenius norm

$$\|A\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n a_{i,j}^2}$$

Weights can be applied by 2 methods, if desired. See the paper for more information.

We will assume our weights are a diagonal matrix, W . Unweighted, we can set $W = I$.

$$\|A\|_W = \|W^{\frac{1}{2}} A W^{\frac{1}{2}}\|_F$$

Higham proposes Algorithm 3.3 (page 9 of the pdf, 337 of the journal). This method alternates projections until the norm converges.

The first projection,

$$P_U(A) = A - W^{-1} \text{diag}(\theta_i) W^{-1}$$

Where θ solves the equation

$$(W^{-1} \# W^{-1}) \theta = \text{diag}(A - I)$$

$\#$ is element wise multiplication.

When W is diagonal (which we are assuming), then

$$P_U(A) = (p_{i,j})$$

$$p_{i,j} = a_{i,j} \quad \forall i \neq j$$

$$p_{i,j} = 1 \quad \forall i = j$$

P_U is the easy projection. It sets the diagonal back to 1 and leaves everything else alone.

The second projection is similar to the spectral decomposition we performed earlier.

$$P_S(A) = W^{\frac{-1}{2}} \left(\left(W^{\frac{1}{2}} A W^{\frac{1}{2}} \right)_+ \right) W^{\frac{-1}{2}}$$

$$(A)_+ = S \text{diag}(\max(\lambda_i, 0)) S^T$$

That is, the eigenvalue system with negative eigenvalues set to 0.

Neither projection alone gives a correlation matrix. P_S makes the matrix PSD and breaks the unit diagonal. P_U fixes the diagonal and can break the PSD property. Alternating between them converges to a matrix in both sets.

The algorithm proceeds:

$$\Delta S_0 = 0, \quad Y_0 = A, \quad \gamma_0 = \max \text{ Float}$$

Loop $k \in 1 \dots \max \text{ Iterations}$

$$R_k = Y_{k-1} - \Delta S_{k-1}$$

$$X_k = P_S(R_k)$$

$$\Delta S_k = X_k - R_k$$

$$Y_k = P_U(X_k)$$

$$\gamma_k = \gamma(Y_k)$$

if $|\gamma_k - \gamma_{k-1}| < \text{tol}$ then break

We use the alternating projections to update the Y matrix until we max out our iterations, or the weighted norm converges to a set tolerance.

The ΔS term is a Dykstra correction. Plain alternating projections converge to some point in the intersection. The correction is what makes the limit the nearest one.

6.3 In Practice

In practice, you need to check the smallest eigenvalue from the result. If it is significantly negative, you need to run the algorithm on the result. You need a result that can be processed by your Cholesky function.

Convergence of the norm is not the same as the answer being usable, and the tolerance you set on γ says nothing directly about the smallest eigenvalue. Check the eigenvalue, not the norm.

7 Principal Component Analysis

PCA provides us another way to simulate a system. The Rebonato and Jackel methodology used to adjust a non-PSD matrix is simply PCA, removing the negative eigenvalues. PCA is often used to reduce the dimensionality of the problem.

PCA projects the system into an orthogonal space. It gives us a linear transform of independent random variables into a correlated multivariate distribution.

7.1 The Decomposition

$$C = S\Lambda S^T$$

Where S are the eigenvectors and Λ is a diagonal matrix of eigenvalues.

If we take the square root of the eigenvalues and store those in a diagonal matrix, $\Lambda^{\frac{1}{2}}$, we can write

$$C = [S\Lambda^{\frac{1}{2}}] [\Lambda^{\frac{1}{2}}S^T]$$

$$B = [S\Lambda^{\frac{1}{2}}] \Rightarrow C = BB^T$$

This provides an alternative to the Cholesky factorization. The requirement has not changed since Section 1.2:

$$X \sim N(\mu, \Sigma)$$

There exists

$$\mu \in \mathbb{R}^n, L \in \mathbb{R}^{n \times m} \quad \text{such that} \quad X = LZ + \mu$$

Where

$$Z_i \sim N(0, 1), \quad i \in [1, m]$$

$$\Sigma = LL'$$

B satisfies that last line, so set $L = B$ and the multivariate draw algorithm runs unchanged. Cholesky and PCA are two ways of answering the same question. Nothing about the simulation cares which one produced the root.

The Cholesky is preferred for a full rank system. Because it is a triangular matrix, fewer floating point operations have to be performed.

7.2 Dimensionality Reduction

If the matrix is PSD then there will be m non-zero eigenvalues where $m \leq n$.

If the eigenvalue is 0, then we can remove it from the Λ matrix, along with the corresponding column in the eigenvector matrix S .

Now to simulate N values, we only have to generate M random numbers, effectively reducing the size of the problem.

This is the same rank deficiency as the zero columns in Section 2.2, handled explicitly rather than patched. Both approaches produce a valid $\Sigma = BB'$. Neither one recovers the information that was never in the matrix.

We can further reduce the dimensionality by removing additional non-zero eigenvalues. The larger the eigenvalue, the more impact it has on the total system.

7.3 Variance Explained

Take the vector of m non-zero eigenvalues. Sort them from largest to smallest (most eigenvalue functions do this for you) to form the vector

$$L = [\lambda_1, \lambda_2, \dots, \lambda_m] \quad \forall \lambda_i \geq \lambda_j, \quad i < j, \quad \text{with } i, j \in [1, 2, \dots, m]$$

Then the percentage of variance explained by the first K eigenvalues is

$$\text{Pct Explained}(K) = \frac{\sum_{i=1}^K L_i}{\sum_{j=1}^m L_j}$$

Dropping eigenvalues is a modeling choice, not a repair. Every dropped component is a direction the simulation can no longer move in, and the forecast variance of any portfolio sitting in that direction is exactly zero. Week 09 shows what that looks like when it goes wrong.

8 Simulating from Models

Often we have a model for an asset. This could be OLS, MLE, ARMA, or something else. At the heart of most modeling is to get to an error variable that is independent and identically distributed (iid).

That is the connection back to Week 02. Every model in that week existed to strip structure out of a series until what was left satisfied the OLS error assumptions. Here we run the same machinery backwards – simulate the iid errors, then push them through the model to rebuild the series.

Assume we have n variables, X , that we are interested in. There is some information matrix, Ω , that we use for modeling, and the error terms, ϵ , are all assumed to be normally distributed.

Models for variable i at time t , $x_{i,t} \in X$, are given as a function:

$$x_{i,t} = f_i(\Omega_t, \epsilon_{i,t})$$

Because we assume $\epsilon_i \sim N(0, \sigma_i^2)$ *iid*, then we can assume the system of errors is jointly normal

$$\epsilon \sim N(0, \Sigma)$$

The models are separate but the errors are not. Two stocks each carrying their own ARMA still move together, and Σ is where that co-movement lives.

If we know Ω_t then we can use the set of functions, f , to simulate X :

1. Simulate m draws for n error terms
2. For each draw, evaluate the set of functions to create the X simulated variables

If we do not know all elements of Ω_t then we need to

1. Simulate them through their own model
2. Assume a value for them

Often some x_i will depend on other elements in X . In that case we need to order the models so that the independent models are evaluated first.

1. Order models without dependencies on X first
2. Order remaining models such that dependent models are evaluated earlier
3. Simulate m draws for n error terms
4. Evaluate the models in order

If you have a circular pattern, that is, A depends on B, which depends on C, which depends on A, then you need an iterative solver. We will not deal with it in this class.